University of Karachi

Department of Computer Science

Bachelor of Science in Computer Science (BSCS)

CS-551 Artificial Intelligence

YELO

AI-Assisted Littering Detection Using YOLOv8 and Multi-Object Tracking

Project Report

Submitted To

Sir Rana Zaeem Tariq

Submitted By

#	Name	Roll Number
1	**Muhammad Ali** (Team Lead)	B23110006087
2	**Muneeb Roshan**	B23110006125
3	**Abdur Rahman**	B23110006005
4	**Arhum Farooq**	B23110006017
5	**Sami Ullah**	B23110006146

Submission Date: 26 July 2026

Group Members and Contributions

#	Name	Roll Number	Role
1	**Muhammad Ali**	B23110006087	Team Lead - Model training and event-detection algorithm design
2	**Muneeb Roshan**	B23110006125	Dataset engineering - selection, class merging, augmentation strategy
3	**Abdur Rahman**	B23110006005	Multi-object tracking - ByteTrack integration and track-identity logic
4	**Arhum Farooq**	B23110006017	Model evaluation - metrics analysis, benchmarking, threshold tuning
5	**Sami Ullah**	B23110006146	Inference pipeline - frame processing, zone geometry, deployment testing

Abstract

This project presents YELO, an AI-assisted system that detects littering events from live camera feeds. Rather than simply classifying whether rubbish is visible in an image, the system determines *when a littering act has occurred* by combining object detection, multi-object tracking, and a rule-based temporal decision layer.

We fine-tuned a YOLOv8-small detector on the TACO (Trash Annotations in Context) dataset using transfer learning and data augmentation, achieving **mAP@0.5 of 0.513** with **precision of 0.713** on the validation split. Detection alone proved insufficient for the task, so we integrated ByteTrack to maintain persistent object identities across frames, and designed a five-condition event rule that requires a waste object to be inside a monitored zone, become stationary, and have been near a tracked person before an incident is raised.

The resulting system runs at approximately 13 frames per second on CPU and produces reviewable incidents with photographic and video evidence. We demonstrate that the temporal decision layer is what makes a detector with moderate recall (0.436) practically usable, since the system observes each scene repeatedly rather than relying on a single frame.

Keywords: object detection, YOLOv8, transfer learning, multi-object tracking, ByteTrack, computer vision, litter detection

Table of Contents

---

1. Introduction

Littering in residential communities is a persistent enforcement problem. Security staff cannot observe every area continuously, and closed-circuit television footage is typically reviewed only after a complaint has been filed, by which time identifying the responsible individual is impractical. Continuous recording of all camera feeds is additionally expensive in storage terms and raises legitimate privacy concerns for residents.

This project addresses the problem with an **event-driven** approach. Camera frames are analysed in memory and discarded; only when the system determines with reasonable confidence that a littering event has occurred is evidence retained. This reduces storage requirements substantially and limits recorded footage to moments of suspected violation.

The core contribution of this work is not the object detector itself, which applies established transfer-learning practice, but the **temporal decision layer** that converts per-frame detections into event determinations. We show that this layer is essential: raw detector output on a realistic scene produces continuous false alarms, whereas the combined system produces reviewable incidents at a rate a human operator can reasonably handle.

1.1 Objectives

1. Train a litter-detection model suitable for outdoor camera imagery.
2. Maintain persistent object identity across frames to enable temporal reasoning.
3. Design and implement a decision rule that distinguishes littering from the mere presence of rubbish.

4. Produce reviewable evidence for each confirmed event.
5. Achieve inference performance adequate for live camera input on commodity hardware without a GPU.

---

2. Problem Statement

The naive formulation of this task is image classification: given a photograph, does it contain litter? This formulation is inadequate for three reasons.

First, it lacks localisation. Knowing that litter exists somewhere in a frame does not indicate whether it lies inside a monitored area. The system must determine position, not merely presence.

Second, it lacks temporal reasoning. A bottle placed in a waste bin and a bottle dropped on a footpath are visually similar objects. What distinguishes littering is the *transition* from an object being carried to being abandoned, which is only observable across time.

Third, it lacks causal association. Rubbish that has been present for hours is not a new littering event. An event requires a person to have been present and to have deposited the object.

We therefore formulate the problem as follows:

> *Given a stream of camera frames, determine the time at which a waste object transitions from being carried by a person to being abandoned within a monitored region, with a false-alarm rate low enough that a human reviewer will act on the system's output.*

This formulation drives the entire system design: detection provides *what and where*, tracking provides *identity over time*, and the event rule provides *when*.

---

3. Literature and Background

3.1 Object detection architectures

Object detection approaches divide broadly into two-stage and single-stage architectures. Two-stage detectors such as Faster R-CNN first generate region proposals and then classify them, generally achieving higher accuracy at significant computational cost. Single-stage detectors such as YOLO (You Only Look Once) predict bounding boxes and class probabilities in a single forward pass.

For this application, single-stage detection is clearly preferable. The system processes a continuous frame stream on CPU hardware, and the accuracy advantage of two-stage detectors does not compensate for their latency in a real-time setting.

We selected **YOLOv8** from the Ultralytics implementation, which provides mature tooling for training, validation, and export, and ships pretrained weights on the COCO dataset.

3.2 Transfer learning

Training a detector from random initialisation requires large datasets and extended training time. Transfer learning instead initialises from weights learned on a large general-purpose dataset, then fine-tunes on the target domain. The pretrained backbone already encodes low-level features such as edges, textures, and object-like shapes, which transfer readily across visual domains.

Given our constraints - a dataset of approximately 3,600 images and access only to a free-tier GPU runtime - transfer learning was the only viable approach.

3.3 Multi-object tracking

Object detection is stateless: each frame is processed independently, and there is no correspondence between detections in consecutive frames. Multi-object tracking establishes this correspondence, assigning persistent identifiers to objects across time.

ByteTrack associates detections with existing tracks using both high-confidence and low-confidence detections, which improves robustness when objects are partially occluded or briefly missed by the detector. This property is directly relevant to our application, where litter is frequently small and intermittently detected.

---

4. Dataset

4.1 Selection

We trained on **TACO (Trash Annotations in Context)**, obtained from Roboflow Universe in YOLOv8 detection format.

Property	Value
Source	Roboflow Universe
Workspace / project	`mohamed-traore-2ekkp/taco-trash-annotations-in-context`
Version	16 (resize-640, 3x augmented, all classes)
Size	approximately 3,597 images
Splits	train / validation / test
Format	YOLOv8 detection
Licence	CC BY 4.0

4.2 Justification

TACO was selected over alternative litter datasets for four reasons.

Contextual photography. TACO images depict rubbish in the environments where it is actually found: pavements, grass verges, gutters, and beaches. Datasets composed of isolated objects photographed against uniform backgrounds produce models that fail when deployed outdoors, because the model has learned to associate the object with a clean background rather than learning the object itself.

Viewpoint correspondence. The majority of TACO images are captured looking downward at ground level from approximately human height. This closely matches the viewpoint of a fixed camera monitoring a footpath.

Adversarial backgrounds. The dataset deliberately includes difficult surfaces - dry leaves, gravel, wet stone, and dappled shadow. These are precisely the textures that generate false positives in deployment. Including them in training teaches the model to reject them.

Provenance and licensing. TACO is a peer-reviewed academic dataset released under CC BY 4.0, making the work reproducible and the dataset redistributable with attribution.

4.3 Class configuration

The published dataset provides fine-grained categories such as plastic bottle, cigarette, aluminium can, and food wrapper. We merged all categories into a **single class,** `trash`, for three reasons.

1. **The application does not consume the sub-type.** An incident record states that littering occurred; it does not require the material classification of the discarded item. Allocating model capacity to a prediction that is then discarded is wasteful.
2. **Severe class imbalance.** Several categories contain only a handful of annotated instances. Retaining them produces near-zero per-class average precision and a misleadingly depressed mean average precision.
3. **Improved examples per class.** Merging substantially increases the number of positive examples the model sees for its single class, which is significant when training for a limited number of epochs.

The trade-off is acknowledged: the system reports *that* litter is present, not *what kind*. For enforcement purposes this is sufficient.

---

5. Methodology

5.1 System architecture

```
Camera client (mobile / webcam)
          |
          |  sampled JPEG frames, approximately 1 per second
          v
Inference gateway (Python)
   1. YOLOv8 detection      -> bounding boxes, labels, confidences
   2. ByteTrack tracking    -> persistent per-object identifiers
   3. Zone evaluation       -> is the object inside a monitored polygon?
   4. Event rule            -> did a littering event occur?
          |
          |  only upon confirmation
          v
Backend: database, private evidence storage, realtime notification
          |
          v
Review dashboard (human-in-the-loop)
```

5.2 Detection model

Property	Value
Architecture	YOLOv8s (small)
Parameters	11.14 million
Initial weights	`yolov8s.pt`, pretrained on COCO
Training method	Transfer learning (fine-tuning)
Output classes	1 (`trash`)

The `s` variant was selected following the benchmarking presented in Section 7.3, which showed it to occupy the practical optimum between accuracy and inference latency on CPU hardware.

5.3 Training configuration

Training was conducted on Google Colab using a free-tier NVIDIA T4 GPU. No group member had access to CUDA-capable local hardware, making CPU training impractical.

```
from ultralytics import YOLO

model = YOLO("yolov8s.pt")

model.train(
    data      = f"{dataset.location}/data.yaml",
    epochs    = 50,
    imgsz     = 640,
    batch     = 16,
    patience  = 15,
    optimizer = "auto",
    lr0       = 0.01,
)
```

Hyperparameter	Value	Justification
`epochs`	50	Validation curves flattened by approximately epoch 45; further training risked overfitting for marginal gain
`imgsz`	640	Framework default; litter objects are small, and reducing resolution causes them to be lost
`batch`	16	Largest batch fitting comfortably within T4 memory at 640 pixels
`patience`	15	Early stopping if fifteen consecutive epochs yield no improvement
`optimizer`	auto	Framework-selected optimiser and schedule
`lr0`	0.01	Framework default; not independently tuned

Training completed all 50 epochs without early stopping being triggered.

5.4 Data augmentation

Data augmentation constitutes the feature-engineering stage for image models. Training images are synthetically varied at each epoch so that the network learns object characteristics rather than memorising specific pixel arrangements. Each transformation corresponds to a realistic deployment condition.

Augmentation	Transformation	Corresponding real-world variation
`hsv_h` , `hsv_s` , `hsv_v`	Hue, saturation, brightness shift	Camera differences, time of day, shade
`degrees`	Small rotation	Camera not perfectly level
`translate`	Positional shift	Litter may appear anywhere in frame
`scale`	Zoom variation	Objects at varying distance
`fliplr`	Horizontal mirror	Litter has no canonical orientation
`mosaic`	Composite of four images	More objects and context per training step
`erasing`	Random region removal	Partial occlusion

5.5 Multi-object tracking

ByteTrack is applied to detector output, maintaining a persistent identifier for each object across frames. Tracker state is maintained **independently per camera**, preventing identity confusion between concurrent feeds, and stale tracks are pruned after a configurable interval.

Tracking is a prerequisite for the event rule: without stable identities, the questions "has this object moved?" and "how long has it been stationary?" cannot be formulated.

5.6 Event determination rule

The system confirms a littering event only when **all five** conditions hold simultaneously.

#	Condition	Parameter	Rationale
1	Detection confidence exceeds threshold	0.30	Filters low-confidence proposals
2	Object lies within an active restricted zone	polygon test	Only monitored areas are relevant
3	A person was tracked nearby recently	0.75 normalised distance, 15 s memory	Rubbish does not appear unaided
4	The object has become stationary	movement < 0.015 normalised	Carried is not equivalent to dropped
5	Stationarity persists for the confirmation delay	5 s, per-camera setting	Rejects single-frame false positives

A confirmed track subsequently enters a **120-second cooldown**, preventing a single item from generating repeated alerts.

The discriminative value of each condition is best illustrated by what it excludes:

Scenario	Outcome	Excluded by
Bottle resting in a waste bin, no person present	No event	Condition 3
Person carrying a bottle across the zone	No event; timer resets on each movement	Condition 4
Single-frame spurious detection	No event	Condition 5
Person deposits bottle and departs	**Event confirmed**	-

5.7 Geometric considerations

Zone membership is evaluated using the **bottom-centre point** of each bounding box, representing the object's contact point with the ground plane, rather than the box centroid.

This distinction is material. The centroid of a standing person's bounding box falls at approximately waist height, which may project outside a ground-plane polygon that the person is demonstrably standing within. Using the ground contact point yields correct zone attribution for both people and objects.

Polygon membership is determined by a ray-casting algorithm implemented directly, avoiding an additional geometry dependency.

---

6. Implementation

6.1 Technology stack

Component	Technology
Detection and training	Ultralytics YOLOv8, PyTorch
Tracking	ByteTrack
Image processing	OpenCV (headless)
Inference service	Python standard library HTTP server
Backend	PostgreSQL, edge functions, object storage
Review interface	Next.js, React
Mobile client	Capacitor (Android)

The inference service deliberately avoids a web framework, using only `http.server` from the Python standard library. This minimises dependencies for a service whose interface comprises two endpoints.

6.2 Inference pipeline

For each received frame:

1. **Validation.** The JPEG signature is verified and the camera's authentication token is validated against the backend.
2. **Buffering.** The frame is retained in a bounded in-memory buffer for potential evidence-clip construction. No frame is written to persistent storage at this stage.
3. **Detection and tracking.** A combined call performs detection and track association, with tracker state persisting between invocations.
4. **Normalisation.** Pixel coordinates are converted to the range 0-1, rendering downstream logic resolution-independent.
5. **Zone evaluation.** The ground contact point is tested against each active zone polygon.
6. **Event evaluation.** The five-condition rule is applied.
7. **Evidence handling.** Upon confirmation, the evidence image, video clip, incident record, and notification are dispatched on background threads so that network latency does not block frame processing.

Frames that do not contribute to a confirmed event are discarded.

6.3 Configurability

Model selection, confidence threshold, inference resolution, tracker configuration, and all event-rule parameters are exposed as environment variables. Consequently, replacing the detection model requires no code modification. This design permitted the project to be developed in two stages (Section 8.1) with the model substituted by configuration alone.

---

7. Results and Evaluation

7.1 Detection performance

Validation metrics, read directly from the trained model checkpoint:

Metric	Value
mAP@0.5	**0.513**
mAP@0.5:0.95	**0.366**
Precision	**0.713**
Recall	**0.436**
Validation box loss	1.044
Validation classification loss	1.157
Validation DFL loss	1.090

7.2 Training behaviour

All six loss components - box, classification, and distribution focal loss, across both training and validation splits - decreased smoothly and monotonically throughout training. No divergence appeared between training and validation losses, indicating that **overfitting did not occur** despite the relatively small dataset. This is attributable to the augmentation strategy described in Section 5.4.

Precision, recall, mAP@0.5, and mAP@0.5:0.95 all increased steadily and began to plateau at approximately epochs 40 to 45, confirming that the chosen training duration was appropriate.

7.3 Inference benchmarking

Models were benchmarked on the development machine (AMD Ryzen, 12 cores, 16 GB RAM, CPU only), reporting the mean of five runs following warm-up.

Model	Size	Input size	Latency	Throughput	Classes
`yolo26n.pt`	5.3 MB	640	103 ms	9.7 FPS	80 (COCO)
`yolo26s.pt`	20 MB	640	216 ms	4.6 FPS	80 (COCO)
`yolo26m.pt`	43 MB	960	1166 ms	0.9 FPS	80 (COCO)
`yolo_garbage.pt`	22 MB	640	127 ms	7.9 FPS	1 (`trash`)
`yolo_garbage.pt`	**22 MB**	**480**	**75 ms**	**13.3 FPS**	**1 (`trash`)**

The deployed configuration - our trained model at 480-pixel input - is both the fastest configuration measured and the only one trained on the target domain. Reducing input resolution from 640 to 480 pixels nearly halves latency; because the gateway samples approximately one frame per second, the modest accuracy reduction on very small objects is an acceptable trade for the additional processing headroom.

The larger COCO models are unsuitable for CPU deployment. At 1.17 seconds per frame, the medium variant processes fewer than one frame per second, at which rate objects displace too far between frames for reliable track association.

7.4 Confusion matrix analysis

	True `trash`	**True `background`**
Predicted `trash`	826	32,620
Predicted `background`	188	-

The background false-positive count requires careful interpretation. In object detection, every proposed bounding box that does not correspond to a ground-truth annotation is counted as a background false positive; this figure is therefore dominated by low-confidence proposals rather than representing 32,620 misclassified images.

The matrix nevertheless characterises the model accurately: it is **liberal in proposing detections**. This directly motivates the system design, in which a confidence threshold and the multi-condition event rule together suppress spurious proposals rather than alerting on raw detector output.

7.5 Qualitative results

On validation imagery the model correctly localises a beverage can on wet rock at 0.82 confidence, and identifies bottles, food wrappers, cardboard packaging, and cigarette ends across pavement, grass, sand, and leaf-litter backgrounds. Confidence for clearly visible objects typically falls between 0.7 and 1.0, while ambiguous instances score between 0.28 and 0.5 - the range in which the configurable threshold is decisive.

---

8. Discussion

8.1 Two-stage development approach

The system was deliberately constructed in two stages.

Stage one established the complete pipeline - capture, detection, tracking, zone evaluation, event determination, evidence handling, and review - using an off-the-shelf COCO-pretrained model, treating the COCO classes `bottle` and `cup` as proxies for litter. This validated the architecture without waiting on model training, but the resulting system was not a litter detector: it could not recognise wrappers, cigarette ends, or crumpled paper, and it detected bottles being actively used rather than discarded.

Stage two substituted the custom model trained on TACO. Because model selection is a configuration parameter, this required changing two configuration lines and no application code.

This staged approach is worth noting as a design outcome: separating the model from the decision logic allowed each to be developed and validated independently.

8.2 The precision-recall trade-off

Our model favours precision (0.713) over recall (0.436). This is the correct orientation for the application, for a reason specific to the temporal design.

A missed detection in a single frame does **not** constitute a missed event. The gateway observes each scene repeatedly, and the event rule requires an object to persist for several seconds regardless. A litter object need only be detected in a sufficient proportion of frames within the confirmation window.

A false positive, by contrast, propagates directly to a human reviewer, consuming their attention and eroding confidence in the system. Trust is the scarce resource in an operator-facing tool.

The temporal decision layer therefore *converts* a moderate-recall detector into an acceptable event detector. This is the principal finding of the project.

8.3 Threshold selection

The confidence threshold was set to **0.30** based on the following reasoning:

- Recall (0.436) is the weaker metric and cannot be further sacrificed; raising the threshold would discard detections the system requires.
- The confusion matrix demonstrates substantial low-confidence noise; reducing the threshold substantially below 0.30 admits this noise into the event logic.
- Genuine litter detections in our validation sampling cluster between 0.4 and 1.0, with harder instances between 0.28 and 0.50. A threshold of 0.30 lies immediately below this band.
- Critically, confidence alone never raises an alert. The four remaining conditions perform the substantive filtering, permitting a permissive confidence threshold that recovers recall which would otherwise be forfeited.

8.4 Privacy considerations

The event-driven design has a privacy consequence worth stating explicitly. Because frames are analysed in memory and discarded unless an event is confirmed, the system retains footage only of moments it assesses as probable violations. It does not constitute continuous surveillance infrastructure.

Additionally, the system performs no face recognition and does not automatically attribute guilt. Confirmed events are presented to a human reviewer who may confirm, reject, or resolve them. This human-in-the-loop requirement is a deliberate design constraint rather than an unimplemented feature.

---

9. Limitations

We state these plainly rather than overstating system capability.

1. **Recall of 0.436.** Small, occluded, or visually camouflaged litter is missed in individual frames. This is partially mitigated by repeated observation of each scene, but a persistently undetected object will not generate an event.
2. **Single-class output.** The system reports the presence of litter, not its material category. Reintroducing fine-grained classes would require substantially more annotated data per category.
3. **CPU inference throughput.** At approximately 13 FPS on 480-pixel input, throughput is adequate for the sampled frames actually processed, but insufficient for full-rate multi-camera video. CUDA-capable hardware would remove this constraint.
4. **Proximity-based person association.** The rule associates the nearest recently-observed person rather than detecting the physical act of discarding. In a crowded scene, an event may be attributed to the wrong individual. Pose estimation would address this directly.

5. **Elevated background false-positive rate in raw detection.** The event rule compensates for this, but the underlying detector remains liberal in its proposals.
6. **No cloud deployment.** Cloud deployment was evaluated but not pursued; free-tier instances provided insufficient memory for the evidence-clip frame buffer, and GPU instances were outside the project's zero-cost constraint. Inference runs on local hardware exposed via a secure tunnel.

---

10. Conclusion and Future Work

10.1 Conclusion

We have presented a working AI-assisted littering detection system. A YOLOv8-small detector was fine-tuned on the TACO dataset using transfer learning and data augmentation, achieving mAP@0.5 of 0.513 and precision of 0.713. ByteTrack was integrated to maintain persistent object identity, and a five-condition event rule was designed to determine whether an observed scene constitutes a littering event rather than merely containing visible rubbish.

The principal finding is that **the temporal decision layer, not detector accuracy alone, determines system usability**. A detector with recall of 0.436 would be inadequate for single-frame classification, yet proves sufficient here because the system observes each scene repeatedly and requires temporal persistence before confirming an event. Separating detection from decision allowed a moderate-accuracy model to support an acceptable end-to-end system.

All stated objectives were met: the model was trained and integrated, tracking maintains identity across frames, the decision rule distinguishes littering from litter presence, evidence is produced for review, and inference performance is adequate for live input on CPU hardware.

10.2 Future work

1. **Improve recall** through extended training, additional augmentation, and supplementary annotated data emphasising small and occluded instances.
2. **Introduce pose estimation** to detect the discarding motion directly, replacing proximity-based person association.
3. **Export to ONNX or TensorRT** for faster inference on CPU and edge devices.
4. **Deploy to GPU-backed infrastructure** for a permanently available service.
5. **Reintroduce fine-grained classification** once sufficient per-category data is available.

---

11. References

1. Proenca, P. F. and Simoes, P. (2020). *TACO: Trash Annotations in Context for Litter Detection*. arXiv:2003.06975. Available at: https://arxiv.org/abs/2003.06975

2. Jocher, G., Chaurasia, A. and Qiu, J. (2023). *Ultralytics YOLOv8*. Available at: https://github.com/ultralytics/ultralytics
3. Zhang, Y. et al. (2022). *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*. European Conference on Computer Vision (ECCV). arXiv:2110.06864.
4. Lin, T.-Y. et al. (2014). *Microsoft COCO: Common Objects in Context*. European Conference on Computer Vision (ECCV). arXiv:1405.0312.
5. Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
6. TACO Dataset. Available at: http://tacodataset.org/
7. Roboflow Universe. *TACO: Trash Annotations in Context*. Available at: https://universe.roboflow.com/mohamed-traore-2ekkp/taco-trash-annotations-in-context

---

12. Appendix

Appendix A: Deliverables

Item	Location
Source code	Project repository
Trained model	`models/yolo_garbage.pt`
Training notebook (with outputs)	`docs/colab_train_garbage.ipynb`
Dataset	See `DATASET.md` for download link
Setup and usage instructions	`README.md`
Dependencies	`requirements.txt`
AI usage declaration	`AI_USAGE_DECLARATION.md`

Appendix B: Model checkpoint metadata

Property	Value
Base architecture	YOLOv8s
Parameters	11,139,000 approximately
Training date	24 July 2026
Framework version	Ultralytics 8.4.104
Output classes	`{0: 'trash'}`
Training dataset	TACO v16
Epochs completed	50 of 50

Appendix C: Deployed configuration

```
YELO_MODEL_PATH=models/yolo_garbage.pt
YELO_MODEL_DEVICE=cpu
YELO_MODEL_CONFIDENCE=0.30
YELO_MODEL_IMAGE_SIZE=480
YELO_WASTE_CLASSES=trash
YELO_TRACKER_CONFIG=bytetrack.yaml
YELO_EVENT_STATIONARY_DISTANCE=0.015
YELO_EVENT_PERSON_DISTANCE=0.75
YELO_EVENT_PERSON_MEMORY_SECONDS=15
YELO_EVENT_CANDIDATE_GRACE_SECONDS=6
YELO_EVENT_COOLDOWN_SECONDS=120
```

Appendix D: Reproducing training

```
# 1. Obtain the dataset
from roboflow import Roboflow
rf = Roboflow(api_key="YOUR_API_KEY")
project = rf.workspace("mohamed-traore-2ekkp").project("taco-trash-annotations-in-context")
dataset = project.version(16).download("yolov8")

# 2. Fine-tune
from ultralytics import YOLO
model = YOLO("yolov8s.pt")
model.train(
    data=f"{dataset.location}/data.yaml",
    epochs=50, imgsz=640, batch=16, patience=15,
)

# 3. Evaluate
metrics = model.val()
print("mAP@0.5      :", metrics.box.map50)
print("mAP@0.5:0.95 :", metrics.box.map)
```

Complete execution with preserved outputs is available in `docs/colab_train_garbage.ipynb`.

End of report.